Convolutional Neural Networks (CNN)
Sıfırdan Sezgisel ve Görsel Bir Anlatım

Atıl Samancıoğlu

1 CNN Nedir? Büyük Resim

CNN (Convolutional Neural Network), özellikle **görsel veriler** üzerinde çalışan bir sinir ağı mimarisidir.

CNN'in temel amacı şudur:

> *Bir görüntünün içindeki anlamlı desenleri (kenar, köşe, şekil, obje) ham piksellerden başlayarak adım adım öğrenmek.*

Buradaki kritik fark şudur:

- Klasik ANN: "Tüm pikselleri düzleştir, hepsini birbirine bağla"
- CNN: "Görüntünün yapısını bozma, yerel desenleri yakala"

CNN, **insan görme sisteminden** ilham alır. Beyindeki görsel korteks de görüntüyü tek seferde anlamaz:

- Önce kenarlar
- Sonra basit şekiller
- Sonra daha karmaşık objeler

CNN de tam olarak bunu yapar.

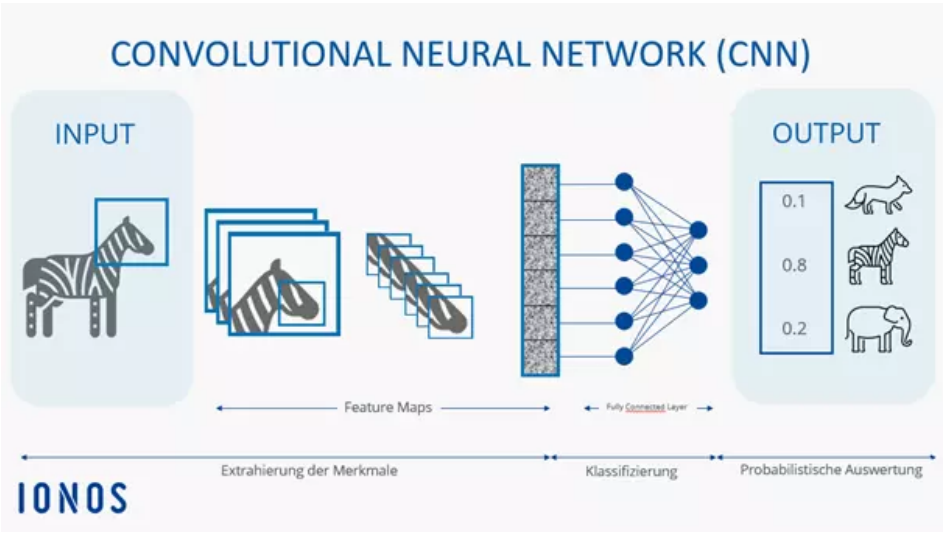

Figure 1: CNN Explained

—

2 CNN Bir Görüntüye Nasıl Bakar?

Bir bilgisayar için görüntü şudur:

- Sayılarla dolu bir matris
- Her sayı bir piksel yoğunluğu

Örneğin bu 6×6 **grayscale** görüntü:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Sezgisel olarak bu şunu temsil eder:

Solda siyah bir alan, sağda beyaz bir alan → ortada güçlü bir dikey kenar var

İnsan bunu anında görür. CNN'in görevi: **bunu matematikle fark etmek**.

—

3 CNN'in Kalbi: Convolution Operasyonu

CNN'in en kritik işlemi: **Convolution**

Convolution şu soruyu sorar:

Görüntünün küçük bir parçası, belirli bir desene benziyor mu?

Bu soruyu sormak için CNN şunu kullanır:

3.1 Kernel (Filter)

Kernel, küçük bir matristir. Örneğin 3×3 bir kernel:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Bu kernel'in sezgisel anlamı şudur:

Soldan sağa geçerken ani bir değişim var mı? Yani dikey kenar var mı?

—

4 Adım Adım Convolution (Takip Edilebilir Örnek)

Şimdi **tek bir adımı** net şekilde takip edelim.

4.1 1. Adım: Kernel Yerleştirme

Kernel, görüntünün sol üst köşesine yerleştirilir:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

4.2 2. Adım: Çarp ve Topla

$$(0 \times 1) + (0 \times 0) + (0 \times -1) + \cdots = 0$$

Sonuç: **0**
Bu, kernel'in bu bölgede aradığı deseni bulamadığı anlamına gelir.
—

4.3 3. Adım: Kaydır (Stride)

Kernel bir adım sağa kayar (stride = 1):
Bu kez kernel, siyah–beyaz sınırına denk gelir.
Sonuç:

$$(0 \times 1) + (0 \times 0) + (1 \times -1) + \cdots = -4$$

Büyük mutlak değer → güçlü bir kenar algılandı

—

5 Stride Nedir?

Stride, kernel'in kaç piksel atlayarak ilerlediğini belirler.

- Stride = 1 → detaylı tarama
- Stride = 2 → daha hızlı, daha az detay

Sezgisel olarak:

Stride büyüdükçe model daha az bakar ama daha hızlı bakar

—

6 Padding Nedir ve Neden Gerekli?

Convolution sonrası çıktı küçülür:

$$6 - 3 + 1 = 4$$

Bu şu anlama gelir:

Kenar bilgisi kayboluyor

Padding çözüm getirir:

- Görüntünün etrafına sıfırlar eklenir
- Çıkış boyutu korunur

$$\text{Same padding} \Rightarrow \text{Giriş boyutu} = \text{Çıkış boyutu}$$

—

7 CNN Neyi Öğrenir?

Önemli nokta:

> **Kernel değerleri sabit değildir.**

Eğitim sırasında CNN:

- Hangi kernel neyi yakalamalı öğrenir
- Kenar mı? Köşe mi? Doku mu?

İlk katmanlar:

- Basit kenarlar

Derin katmanlar:

- Göz, burun, tekerlek, yüz

—

8 Burada Dur ve Görselleştir

Bu noktada **mutlaka** şu interaktif aracı açmanı öneririm:

> **CNN Explainer** `https://poloclub.github.io/cnn-explainer/`

Burada şunları canlı görebilirsin:

- Kernel'in görüntü üzerinde nasıl gezdiğini
- Her adımda çıkan sayıları
- Feature map'lerin nasıl oluştuğunu

Bu görsel deneyim, bu dökümanda okuduklarını **kalıcı hale getirir**.

—

9 Özet

- CNN, görüntüyü parça parça inceler
- Kernel = desen arayıcı
- Convolution = "burada bu desen var mı?" sorusu
- Stride = ne kadar hızlı gezdiğin
- Padding = kenar bilgisini koruma yöntemi

> *CNN, görüntüyü ezberlemez. Desenleri öğrenir.*

Bir sonraki adımda:

- Pooling
- Çok kanallı (RGB) convolution
- Feature map mantığı

inceleyeceğiz.